Introducción a la teoría de NP-completitud

Programación III - UNGS

Problemas de decisión

- Un problema de decisión es un problema cuya respuesta es un valor de tipo **boolean**.

Problemas de decisión

- Un problema de decisión es un problema cuya respuesta es un valor de tipo **boolean**.
- Por ejemplo, el problema de sumar los elementos de un arreglo no es un problema de decisión.

Problemas de decisión

- Un problema de decisión es un problema cuya respuesta es un valor de tipo **boolean**.
- Por ejemplo, el problema de sumar los elementos de un arreglo no es un problema de decisión.
- Sin embargo, se puede convertir en un problema de decisión del siguiente modo:

Problemas de decisión

- Un problema de decisión es un problema cuya respuesta es un valor de tipo **boolean**.
- Por ejemplo, el problema de sumar los elementos de un arreglo no es un problema de decisión.
- Sin embargo, se puede convertir en un problema de decisión del siguiente modo:

  Suma de un arreglo

Problemas de decisión

- Un problema de decisión es un problema cuya respuesta es un valor de tipo **boolean**.
- Por ejemplo, el problema de sumar los elementos de un arreglo no es un problema de decisión.
- Sin embargo, se puede convertir en un problema de decisión del siguiente modo:

SUMA DE UN ARREGLO

1. **Entrada:** Un arreglo A y un número real $k \in \mathbb{R}$.

Problemas de decisión

- Un problema de decisión es un problema cuya respuesta es un valor de tipo **boolean**.
- Por ejemplo, el problema de sumar los elementos de un arreglo no es un problema de decisión.
- Sin embargo, se puede convertir en un problema de decisión del siguiente modo:

  Suma de un arreglo
  1. **Entrada:** Un arreglo A y un número real $k \in \mathbb{R}$.
  2. **Salida:** ¿Es cierto que la suma de los elementos de A es mayor o igual que k?

Problemas de decisión

- Un problema de decisión es un problema cuya respuesta es un valor de tipo **boolean**.
- Por ejemplo, el problema de sumar los elementos de un arreglo no es un problema de decisión.
- Sin embargo, se puede convertir en un problema de decisión del siguiente modo:

  SUMA DE UN ARREGLO
  1. **Entrada:** Un arreglo A y un número real $k \in \mathbb{R}$.
  2. **Salida:** ¿Es cierto que la suma de los elementos de A es mayor o igual que k?

- Utilizando búsqueda binaria sobre k, se puede convertir un algoritmo polinomial para el problema de decisión en un algoritmo polinomial para el problema original.

Problemas NP-completos

- Para muchos problemas no se conocen algoritmos eficientes que los resuelvan en forma exacta.

Problemas NP-completos

- Para muchos problemas no se conocen algoritmos eficientes que los resuelvan en forma exacta.
- La clase de problemas más famosa relacionada con esta problemática es la clase de problemas NP.

Problemas NP-completos

- Para muchos problemas no se conocen algoritmos eficientes que los resuelvan en forma exacta.
- La clase de problemas más famosa relacionada con esta problemática es la clase de problemas NP.
- **Definición.** Una computadora no determinística es una computadora que incluye una operación **guess**(A), que genera una opción aleatoriamente, seleccionada del conjunto A.

Problemas NP-completos

- Para muchos problemas no se conocen algoritmos eficientes que los resuelvan en forma exacta.
- La clase de problemas más famosa relacionada con esta problemática es la clase de problemas NP.
- **Definición.** Una computadora no determinística es una computadora que incluye una operación **guess**(A), que genera una opción aleatoriamente, seleccionada del conjunto A.
- Podemos escribir algoritmos no determinísticos usando esta función.

Problemas NP-completos

- **Ejemplo:** Un algoritmo no determinístico para el problema de camino mínimo.

Problemas NP-completos

- **Ejemplo:** Un algoritmo no determinístico para el problema de camino mínimo.

```
1  Camino minimo(Grafo grafo, int s, int t)
2  {
3     Camino P := new Camino(grafo);
4     int actual = s;
5
6     while (actual != t)
7     {
8        P.add(actual);
9        actual = guess(grafo.vecinos(actual));
10    }
11
12    return P;
13 }
```

Problemas NP-completos

- **Ejemplo:** Un algoritmo no determinístico para la versión de decisión de camino mínimo.

Problemas NP-completos

- **Ejemplo:** Un algoritmo no determinístico para la versión de decisión de camino mínimo.

```
1  boolean caminoMinimo(Grafo grafo, int s, int t, double k)
2  {
3     Camino P := new Camino(grafo);
4     int actual = s;
5
6     while (actual != t)
7     {
8        P.add(actual);
9        actual = guess(grafo.vecinos(actual));
10    }
11
12    return P.distancia() <= k;
13 }
```

Problemas NP-completos

- Decimos que un algoritmo no determinístico resuelve un problema de decisión si existe una secuencia de resultados de **guess()** que retorna **true** si y sólo si la respuesta correcta es afirmativa.

Problemas NP-completos

- Decimos que un algoritmo no determinístico resuelve un problema de decisión si existe una secuencia de resultados de **guess()** que retorna **true** si y sólo si la respuesta correcta es afirmativa.

  1. No siempre tiene que dar la respuesta correcta! Solamente debe existir una ejecución que retorne **true** cuando la respuesta es afirmativa.

Problemas NP-completos

- Decimos que un algoritmo no determinístico resuelve un problema de decisión si existe una secuencia de resultados de **guess()** que retorna **true** si y sólo si la respuesta correcta es afirmativa.

  1. No siempre tiene que dar la respuesta correcta! Solamente debe existir una ejecución que retorne **true** cuando la respuesta es afirmativa.
  2. Si la respuesta correcta es **false**, entonces siempre debe retornar **false**.

Problemas NP-completos

- Decimos que un algoritmo no determinístico resuelve un problema de decisión si existe una secuencia de resultados de **guess()** que retorna **true** si y sólo si la respuesta correcta es afirmativa.

  1. No siempre tiene que dar la respuesta correcta! Solamente debe existir una ejecución que retorne **true** cuando la respuesta es afirmativa.
  2. Si la respuesta correcta es **false**, entonces siempre debe retornar **false**.

- **Definición.** La clase NP está compuesta por los problemas que se pueden resolver en tiempo polinomial por un algoritmo no determinístico.

Problemas NP-completos

- Decimos que un algoritmo no determinístico resuelve un problema de decisión si existe una secuencia de resultados de **guess()** que retorna **true** si y sólo si la respuesta correcta es afirmativa.

  1. No siempre tiene que dar la respuesta correcta! Solamente debe existir una ejecución que retorne **true** cuando la respuesta es afirmativa.
  2. Si la respuesta correcta es **false**, entonces siempre debe retornar **false**.

- **Definición.** La clase NP está compuesta por los problemas que se pueden resolver en tiempo polinomial por un algoritmo no determinístico.

  1. Esto no implica que un problema NP se pueda resolver en tiempo eficiente!

Problemas NP-completos

- Decimos que un algoritmo no determinístico resuelve un problema de decisión si existe una secuencia de resultados de **guess()** que retorna **true** si y sólo si la respuesta correcta es afirmativa.

  1. No siempre tiene que dar la respuesta correcta! Solamente debe existir una ejecución que retorne **true** cuando la respuesta es afirmativa.
  2. Si la respuesta correcta es **false**, entonces siempre debe retornar **false**.

- **Definición.** La clase NP está compuesta por los problemas que se pueden resolver en tiempo polinomial por un algoritmo no determinístico.

  1. Esto no implica que un problema NP se pueda resolver en tiempo eficiente!
  2. Los algoritmos determinísticos también son no determinísticos, con lo cual los problemas polinomiales están en NP.

Problemas NP-completos

- Por ejemplo, la versión de decisión del TSP está en NP:

Problemas NP-completos

- Por ejemplo, la versión de decisión del TSP está en NP:

```
1  boolean tsp(Grafo grafo, double k)
2  {
3    Circuito C := new Circuito(grafo);
4    ArrayList<Integer> todos = grafo.vertices();
5
6    while (!todos.vacio())
7    {
8      int prox = guess(todos);
9      todos.remove(prox);
10     C.add(prox);
11   }
12
13   return C.distancia() <= k;
14 }
```

Problemas NP-completos

- El problema de resolver un sudoku también está en NP:

Problemas NP-completos

- El problema de resolver un sudoku también está en NP:

```
1  boolean sudoku(Matriz matriz)
2  {
3    for(int i=0; i<matriz.filas(); ++i)
4    for(int j=0; j<matriz.columnas(); ++j) if( matriz.libre(i,j) )
5    {
6      int valor = guess({1,...,9});
7      matriz.set(i, j, valor);
8    }
9
10   return matriz.valida();
11 }
```

Problemas NP-completos

- Otros problemas de la clase NP están dados por las versiones de decisión de los siguientes problemas:

Problemas NP-completos

- Otros problemas de la clase NP están dados por las versiones de decisión de los siguientes problemas:

  1. Buscar un elemento en un arreglo.

Problemas NP-completos

- Otros problemas de la clase NP están dados por las versiones de decisión de los siguientes problemas:

  1. Buscar un elemento en un arreglo.
  2. Determinar si un arreglo está ordenado.

Problemas NP-completos

- Otros problemas de la clase NP están dados por las versiones de decisión de los siguientes problemas:

  1. Buscar un elemento en un arreglo.
  2. Determinar si un arreglo está ordenado.
  3. Elemento máximo de un arreglo.

Problemas NP-completos

- Otros problemas de la clase NP están dados por las versiones de decisión de los siguientes problemas:

  1. Buscar un elemento en un arreglo.
  2. Determinar si un arreglo está ordenado.
  3. Elemento máximo de un arreglo.
  4. Determinar si un grafo es conexo.

Problemas NP-completos

- Otros problemas de la clase NP están dados por las versiones de decisión de los siguientes problemas:

  1. Buscar un elemento en un arreglo.
  2. Determinar si un arreglo está ordenado.
  3. Elemento máximo de un arreglo.
  4. Determinar si un grafo es conexo.
  5. Camino mínimo entre dos ciudades.

Problemas NP-completos

- Otros problemas de la clase NP están dados por las versiones de decisión de los siguientes problemas:

  1. Buscar un elemento en un arreglo.
  2. Determinar si un arreglo está ordenado.
  3. Elemento máximo de un arreglo.
  4. Determinar si un grafo es conexo.
  5. Camino mínimo entre dos ciudades.
  6. Camino máximo entre dos ciudades.

Problemas NP-completos

- Otros problemas de la clase NP están dados por las versiones de decisión de los siguientes problemas:

1. Buscar un elemento en un arreglo.
2. Determinar si un arreglo está ordenado.
3. Elemento máximo de un arreglo.
4. Determinar si un grafo es conexo.
5. Camino mínimo entre dos ciudades.
6. Camino máximo entre dos ciudades.
7. Árbol generador mínimo.

Problemas NP-completos

- Otros problemas de la clase NP están dados por las versiones de decisión de los siguientes problemas:

1. Buscar un elemento en un arreglo.
2. Determinar si un arreglo está ordenado.
3. Elemento máximo de un arreglo.
4. Determinar si un grafo es conexo.
5. Camino mínimo entre dos ciudades.
6. Camino máximo entre dos ciudades.
7. Árbol generador mínimo.
8. Flujo máximo.

Problemas NP-completos

- Otros problemas de la clase NP están dados por las versiones de decisión de los siguientes problemas:

1. Buscar un elemento en un arreglo.
2. Determinar si un arreglo está ordenado.
3. Elemento máximo de un arreglo.
4. Determinar si un grafo es conexo.
5. Camino mínimo entre dos ciudades.
6. Camino máximo entre dos ciudades.
7. Árbol generador mínimo.
8. Flujo máximo.
9. Clique máxima.

Problemas NP-completos

- Otros problemas de la clase NP están dados por las versiones de decisión de los siguientes problemas:

  1. Buscar un elemento en un arreglo.
  2. Determinar si un arreglo está ordenado.
  3. Elemento máximo de un arreglo.
  4. Determinar si un grafo es conexo.
  5. Camino mínimo entre dos ciudades.
  6. Camino máximo entre dos ciudades.
  7. Árbol generador mínimo.
  8. Flujo máximo.
  9. Clique máxima.

- Algunos elementos de esta lista se pueden resolver por medio de algoritmos polinomiales (¿cuáles?).

Transformaciones y reducciones

- Consideremos dos problemas de decisión A y B (es decir, problemas cuya solución es **true** o **false**).

Transformaciones y reducciones

- Consideremos dos problemas de decisión A y B (es decir, problemas cuya solución es **true** o **false**).
- **Definición:** Una transformación de A en B es un programa p que toma un input I para A y lo convierte en un input $p(I) = I'$ para B, de modo tal que $A(I) =$ **true** si y sólo si $B(I') =$ **true**.

Transformaciones y reducciones

- Consideremos dos problemas de decisión A y B (es decir, problemas cuya solución es **true** o **false**).
- **Definición:** Una transformación de A en B es un programa p que toma un input I para A y lo convierte en un input $p(I) = I'$ para B, de modo tal que $A(I) =$ **true** si y sólo si $B(I') =$ **true**.
- Si p es un algoritmo polinomial en el tamaño de I, decimos que es una transformación polinomial.

Transformaciones y reducciones

- Consideremos dos problemas de decisión A y B (es decir, problemas cuya solución es **true** o **false**).
- **Definición:** Una transformación de A en B es un programa p que toma un input I para A y lo convierte en un input $p(I) = I'$ para B, de modo tal que $A(I) =$ **true** si y sólo si $B(I') =$ **true**.
- Si p es un algoritmo polinomial en el tamaño de I, decimos que es una transformación polinomial.
- Si existe una transformación polinomial de A en B, escribimos $A \leq_P B$.

Transformaciones y reducciones

- La notación $A \leq_P B$ denota que B es igual o más difícil que A, dado que se puede usar un algoritmo b que resuelve B para resolver A:

Transformaciones y reducciones

- La notación $A \leq_P B$ denota que B es igual o más difícil que A, dado que se puede usar un algoritmo b que resuelve B para resolver A:

  1. Dado el input I, calcular $I' = p(I)$.

Transformaciones y reducciones

- La notación $A \leq_P B$ denota que B es igual o más difícil que A, dado que se puede usar un algoritmo b que resuelve B para resolver A:

  1. Dado el input I, calcular $I' = p(I)$.
  2. Aplicar el algoritmo b sobre I' y retornar $b(I')$.

Transformaciones y reducciones

- La notación $A \leq_P B$ denota que B es igual o más difícil que A, dado que se puede usar un algoritmo b que resuelve B para resolver A:

  1. Dado el input I, calcular $I' = p(I)$.
  2. Aplicar el algoritmo b sobre I' y retornar $b(I')$.

- Se trata de una relación de dificultad entre problemas. Los problemas NP-completos son los problemas más difíciles dentro de NP, de acuerdo con esta relación de dificultad.

Transformaciones y reducciones

- La notación $A \leq_P B$ denota que B es igual o más difícil que A, dado que se puede usar un algoritmo b que resuelve B para resolver A:

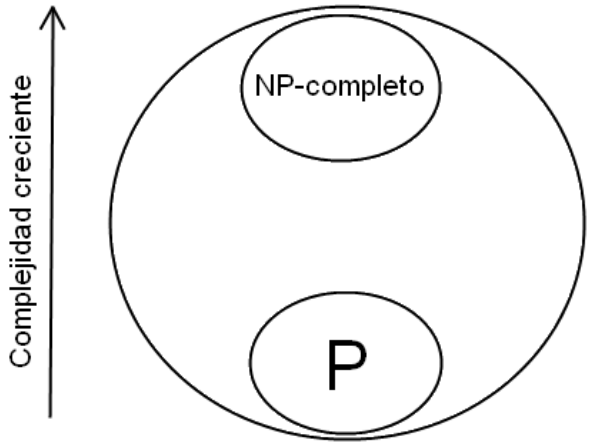
  1. Dado el input I, calcular $I' = p(I)$.
  2. Aplicar el algoritmo b sobre I' y retornar $b(I')$.

- Se trata de una relación de dificultad entre problemas. Los problemas NP-completos son los problemas más difíciles dentro de NP, de acuerdo con esta relación de dificultad.

- **Definición:** Un problema $B \in NP$ es NP-completo si $A \leq_P B$ para todo otro problema $A \in NP$.

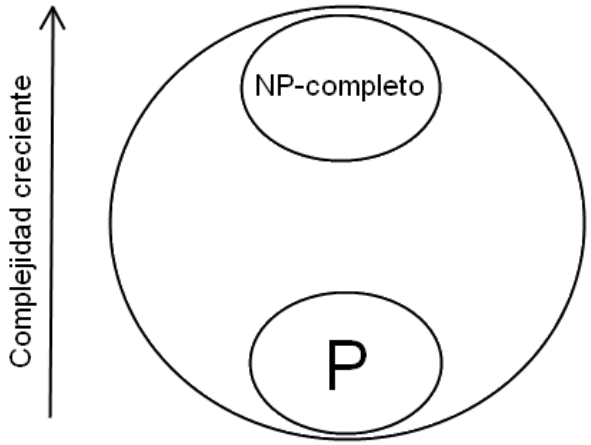
Problemas NP-completos

- Si llamamos P a la clase de problemas que admiten un algoritmo polinomial, la clase NP tiene la siguiente forma:



Problemas NP-completos

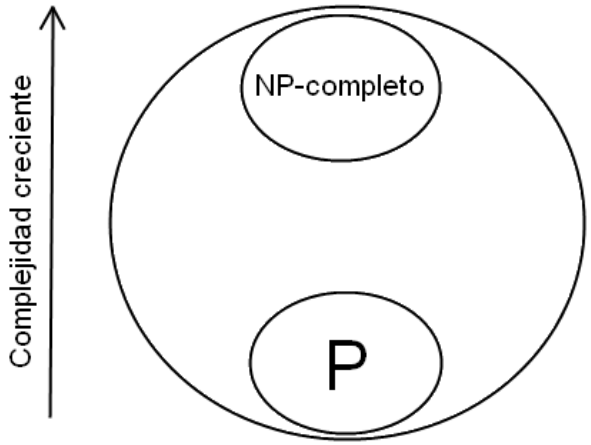
- Si llamamos P a la clase de problemas que admiten un algoritmo polinomial, la clase NP tiene la siguiente forma:



- Los problemas NP-completos son muy difíciles! No se conocen algoritmos polinomiales para resolver estos problemas.

Problemas NP-completos

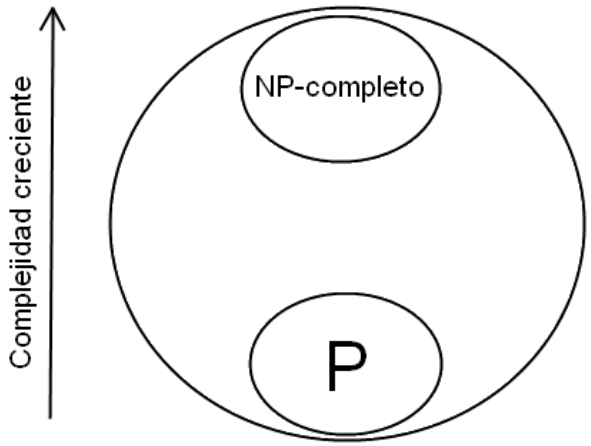
- Si llamamos P a la clase de problemas que admiten un algoritmo polinomial, la clase NP tiene la siguiente forma:



- Los problemas NP-completos son muy difíciles! No se conocen algoritmos polinomiales para resolver estos problemas.
- Este problema abierto se denomina la conjetura P vs. NP.

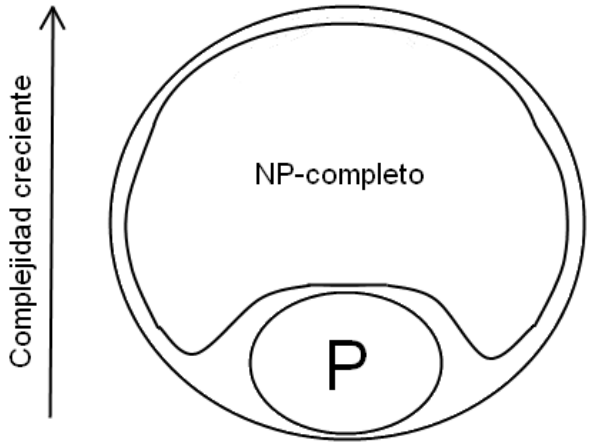
Problemas NP-completos

- La clase NP está muy estudiada. La situación actual no está bien representada por este dibujo:



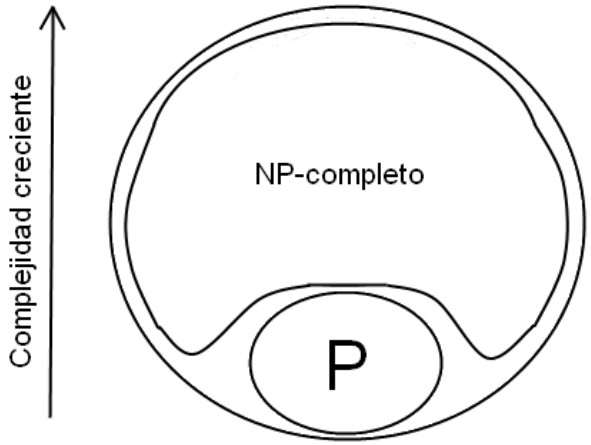
Problemas NP-completos

- Al contrario, con el paso de los años, se probó que la gran mayoría de problemas en NP es NP-completo.



Problemas NP-completos

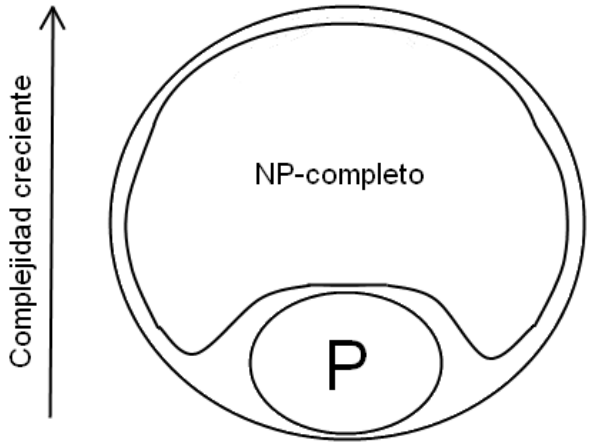
- Al contrario, con el paso de los años, se probó que la gran mayoría de problemas en NP es NP-completo.



- Quedan actualmente muy pocos problemas abiertos en NP, para los cuales no se sabe si están en P o bien son NP-completos.

Problemas NP-completos

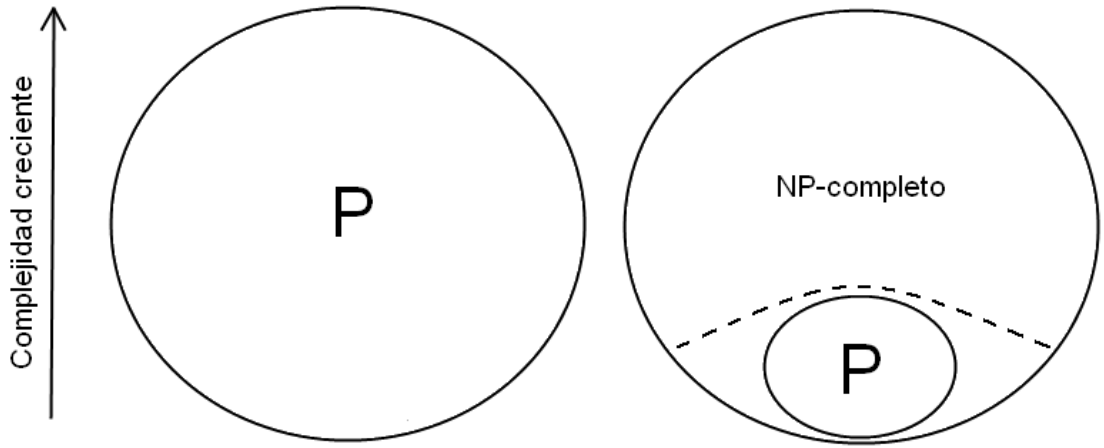
- Al contrario, con el paso de los años, se probó que la gran mayoría de problemas en NP es NP-completo.



- Quedan actualmente muy pocos problemas abiertos en NP, para los cuales no se sabe si están en P o bien son NP-completos.
- Por ejemplo, uno de ellos es determinar la factorización en primos de un número entero.

Problemas NP-completos

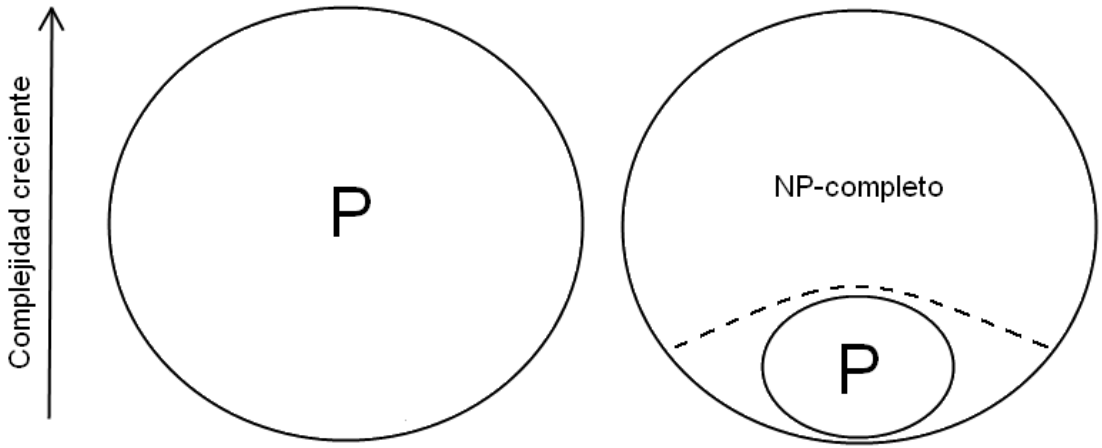
- Existen dos escenarios posibles:



Problemas NP-completos

- Existen dos escenarios posibles:

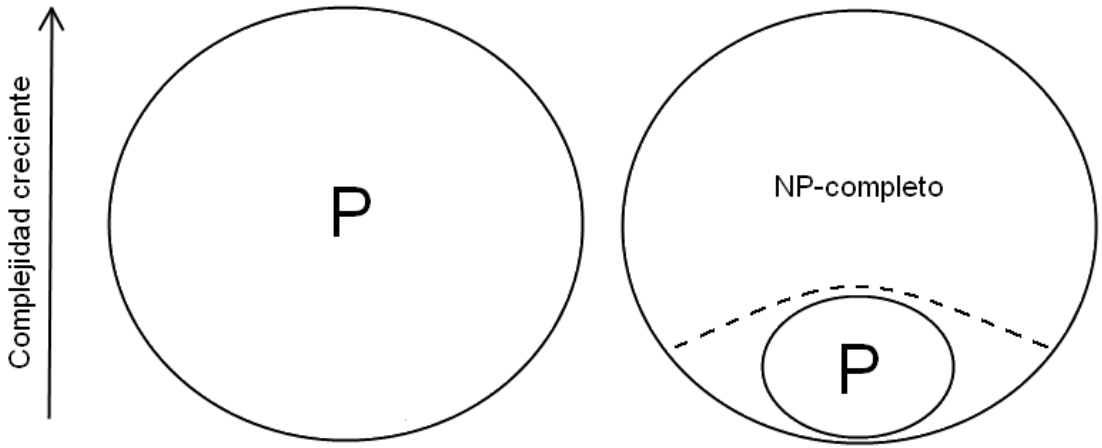
  1. P = NP y entonces todos los problemas en NP se pueden resolver en tiempo polinomial.



Problemas NP-completos

- Existen dos escenarios posibles:

  1. P = NP y entonces todos los problemas en NP se pueden resolver en tiempo polinomial.
  2. P ≠ NP y entonces los problemas de NP son intrínsecamente difíciles.



Problemas NP-completos

- Determinar cuál de las dos situaciones ocurre es uno de los problemas del milenio, con U$S 1,000,000 de premio.

Problemas NP-completos

- Determinar cuál de las dos situaciones ocurre es uno de los problemas del milenio, con U$S 1,000,000 de premio.
- Se trata de uno de los problemas abiertos más importantes de la computación.



Problemas NP-completos

- Algunos problemas NP-completos:

Problemas NP-completos

- Algunos problemas NP-completos:
  1. Clique máxima en grafos.

Problemas NP-completos

- Algunos problemas NP-completos:

  1. Clique máxima en grafos.
  2. Problema del viajante de comercio.

Problemas NP-completos

- Algunos problemas NP-completos:

  1. Clique máxima en grafos.
  2. Problema del viajante de comercio.
  3. Problema de la mochila.

Problemas NP-completos

- Algunos problemas NP-completos:

  1. Clique máxima en grafos.
  2. Problema del viajante de comercio.
  3. Problema de la mochila.
  4. Problema de ruteo de vehículos.

Problemas NP-completos

- Algunos problemas NP-completos:

1. Clique máxima en grafos.
2. Problema del viajante de comercio.
3. Problema de la mochila.
4. Problema de ruteo de vehículos.
5. *Set partition*.

Problemas NP-completos

- Algunos problemas NP-completos:

1. Clique máxima en grafos.
2. Problema del viajante de comercio.
3. Problema de la mochila.
4. Problema de ruteo de vehículos.
5. *Set partition*.
6. Programación de máquinas homogéneas.

Problemas NP-completos

- Algunos problemas NP-completos:

1. Clique máxima en grafos.
2. Problema del viajante de comercio.
3. Problema de la mochila.
4. Problema de ruteo de vehículos.
5. *Set partition*.
6. Programación de máquinas homogéneas.
7. Sudoku.

Problemas NP-completos

- Algunos problemas NP-completos:

1. Clique máxima en grafos.
2. Problema del viajante de comercio.
3. Problema de la mochila.
4. Problema de ruteo de vehículos.
5. *Set partition*.
6. Programación de máquinas homogéneas.
7. Sudoku.
8. Tetris.

Problemas NP-completos

- Algunos problemas NP-completos:

1. Clique máxima en grafos.
2. Problema del viajante de comercio.
3. Problema de la mochila.
4. Problema de ruteo de vehículos.
5. *Set partition*.
6. Programación de máquinas homogéneas.
7. Sudoku.
8. Tetris.
9. SAT: Determinar si una fórmula proposicional es satisfactible.

Problemas NP-completos

- Algunos problemas NP-completos:

1. Clique máxima en grafos.
2. Problema del viajante de comercio.
3. Problema de la mochila.
4. Problema de ruteo de vehículos.
5. *Set partition*.
6. Programación de máquinas homogéneas.
7. Sudoku.
8. Tetris.
9. SAT: Determinar si una fórmula proposicional es satisfactible.
10. Coloreo de grafos.

Problemas NP-completos

- Algunos problemas NP-completos:

1. Clique máxima en grafos.
2. Problema del viajante de comercio.
3. Problema de la mochila.
4. Problema de ruteo de vehículos.
5. *Set partition*.
6. Programación de máquinas homogéneas.
7. Sudoku.
8. Tetris.
9. SAT: Determinar si una fórmula proposicional es satisfactible.
10. Coloreo de grafos.
11. ...

Algoritmos exactos vs. heurísticas

- Dado un problema NP-completo, ¿qué alternativas tenemos?

	Algoritmos exactos	Algoritmos heurísticos
Calidad de la solución	"Buena"	"Mala"
Complejidad	"Mala"	"Buena"

Algoritmos exactos vs. heurísticas

- Dado un problema NP-completo, ¿qué alternativas tenemos?

	Algoritmos exactos	Algoritmos heurísticos
Calidad de la solución	"Buena"	"Mala"
Complejidad	"Mala"	"Buena"

- ¿Podemos tener en un mismo algoritmo una buena complejidad y una buena calidad de solución? No! A menos que P = NP.

Algoritmos exactos vs. heurísticas

- Dado un problema NP-completo, ¿qué alternativas tenemos?

	Algoritmos exactos	Algoritmos heurísticos
Calidad de la solución	"Buena"	"Mala"
Complejidad	"Mala"	"Buena"

- ¿Podemos tener en un mismo algoritmo una buena complejidad y una buena calidad de solución? No! A menos que P = NP.

  1. Podemos tener algoritmos exactos con tiempos razonables,

Algoritmos exactos vs. heurísticas

- Dado un problema NP-completo, ¿qué alternativas tenemos?

	Algoritmos exactos	Algoritmos heurísticos
Calidad de la solución	"Buena"	"Mala"
Complejidad	"Mala"	"Buena"

- ¿Podemos tener en un mismo algoritmo una buena complejidad y una buena calidad de solución? No! A menos que P = NP.

  1. Podemos tener algoritmos exactos con tiempos razonables,
  2. o bien heurísticas con la mejor calidad de solución posible.